

Лабораторна робота №4

Сластнікової Анастасії

Тема: Розширення можливостей PostgreSQL: користувацькі типи, функції та тригери

Мета роботи: Закріпити знання з розширюваності PostgreSQL. Навчитися створювати користувацькі типи даних. Реалізувати власну користувацьку функцію або агрегат. Створити тригери для логування змін у базі даних. Автоматично оновлювати пов'язані таблиці чи заповнювати значення. Оновити діаграму бази даних відповідно до виконаних завдань. Перевірити коректність роботи реалізованих об'єктів через виконання тестових SQL-запитів.

Створення користувацького типу даних

Для онлайн-школи логічно додати тип ENUM для статусів навчання, щоб обмежити можливі варіанти значень у таблиці записів.



```
Query  Query History  Scratch Pad x
1  -- 1. Створення типу
2  CREATE TYPE enrollment_status AS ENUM ('active', 'completed', 'dropped', 'failed');
3
4  -- 2. Додавання стовпця до таблиці enrollments
5  ALTER TABLE enrollments ADD COLUMN status enrollment_status DEFAULT 'active';
6
7  -- 3. Оновлення даних на основі старого стовпця
8  UPDATE enrollments SET status = 'completed' WHERE is_completed = TRUE;
```

Data Output Messages Notifications

UPDATE 4

Query returned successfully in 54 msec.

Створення користувацької функції

Реалізуємо функцію, яка розраховує персональну знижку для студента на основі його середнього балу.

Travel agency/postgres@PostgreSQL 18

Query Query History Scratch Pad x

```

1 CREATE OR REPLACE FUNCTION calculate_student_discount(student_id_param INT)
2 RETURNS NUMERIC AS $$
3 DECLARE
4     avg_grade NUMERIC;
5 BEGIN
6     SELECT AVG(final_grade) INTO avg_grade
7     FROM Enrollments
8     WHERE student_id = student_id_param;
9
10    IF avg_grade >= 95 THEN RETURN 20.0; -- 20% знижки
11    ELSIF avg_grade >= 85 THEN RETURN 10.0; -- 10% знижки
12    ELSE RETURN 0.0;
13    END IF;
14 END;
15 $$ LANGUAGE plpgsql;
16
17 -- Тестовий запит
18 SELECT full_name, calculate_student_discount(student_id) as discount_percentage
19 FROM Students;

```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	full_name character varying (100)	discount_percentage numeric
1	Анастасія	10.0
2	Максим	0.0
3	Анастасія Ковальчук	10.0
4	Марко Шевченко	0.0
5	Дмитро Бондар	0.0

Створення тригерів для логування та автоматизації

Створимо систему аудиту для таблиці Students та автоматичне оновлення статусів.

Query Query History Scratch Pad x

```

1 -- Таблиця для логів
2 CREATE TABLE students_log (
3     log_id SERIAL PRIMARY KEY,
4     student_id INT,
5     operation VARCHAR(10),
6     changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
7     changed_by VARCHAR(50) DEFAULT CURRENT_USER
8 );
9
10 -- Функція тригера
11 CREATE OR REPLACE FUNCTION log_student_changes() RETURNS TRIGGER AS $$
12 BEGIN
13     IF (TG_OP = 'DELETE') THEN
14         INSERT INTO students_log (student_id, operation) VALUES (OLD.student_id, 'DELETE');
15         RETURN OLD;
16     ELSE
17         INSERT INTO students_log (student_id, operation) VALUES (NEW.student_id, TG_OP);
18         RETURN NEW;
19     END IF;
20 END;
21 $$ LANGUAGE plpgsql;
22
23 -- Прив'язка тригера
24 CREATE TRIGGER trg_students_audit
25 AFTER INSERT OR UPDATE OR DELETE ON Students
26 FOR EACH ROW EXECUTE FUNCTION log_student_changes();

```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 92 msec.

Логування змін

The screenshot displays a SQL IDE interface. The main editor shows a query with 12 lines of SQL code. The code defines a function `auto_complete_enrollment()` that returns a trigger, and a trigger `trg_auto_complete` that fires before updates on the `Enrollments` table, executing the function for each row. The IDE has tabs for 'Query', 'Query History', and 'Scratch Pad'. A small pop-up menu is visible over the code, showing options like 'Aa' and '*'. Below the editor, the 'Messages' pane shows the message 'CREATE TRIGGER' and 'Query returned successfully in 49 msec.'

```
1 CREATE OR REPLACE FUNCTION auto_complete_enrollment() RETURNS TRIGGER
2 BEGIN
3     IF NEW.final_grade >= 60 THEN
4         NEW.status = 'completed';
5     END IF;
6     RETURN NEW;
7 END;
8 $$ LANGUAGE plpgsql;
9
10 CREATE TRIGGER trg_auto_complete
11 BEFORE UPDATE ON Enrollments
12 FOR EACH ROW EXECUTE FUNCTION auto_complete_enrollment();
```

CREATE TRIGGER

Query returned successfully in 49 msec.

Автоматичне оновлення

Тригер, який автоматично встановлює статус 'completed', якщо оцінка стає вищою за 60.

Тестові запити для перевірки об'єктів

Перевірка функції знижок

Цей запит показує, як працює функція в реальному часі для кожного студента.

Query

Query History

Scratch Pad

1

SELECT

2

full_name,

3

email,

4

calculate_student_discount(student_id) AS discount_pct

5

FROM students;

Aa

.

*

↑

↓

×

Data Output

Messages

Notifications

Showing rows: 1 to 5

Page No: 1

	full_name character varying (100)	email character varying (100)	discount_pct numeric
1	Анастасія	ana@email.com	10.0
2	Максим	max@email.com	0.0
3	Анастасія Ковальчук	nastya@example.com	10.0
4	Марко Шевченко	mark@example.com	0.0
5	Дмитро Бондар	dima@example.com	0.0

Перевірка тригера логування

Query

Query History

Scratch Pad

1

-- 1. Оновлюємо ім'я студента (наприклад, з ID = 1)

2

UPDATE students SET full_name = 'Анастасія Оновлена' WHERE student_id = 1;

3

-- 2. Перевіряємо таблицю логів

4

SELECT * FROM students_log;

5

Aa

.

*

↑

↓

×

Data Output

Messages

Notifications

Showing rows: 1 to 1

	log_id [PK] integer	student_id integer	operation character varying (10)	changed_at timestamp without time zone	changed_by character varying (50)
1	1	1	UPDATE	2026-04-15 00:44:36.874912	postgres

Перевірка нових статусів

Перевіримо, чи правильно відображається наш користувацький тип ENUM.

Query

Query History

Scratch F

1

```
SELECT enrollment_id, student_id, status FROM enrollments;
```

Aa

*

↑

↓

×

Data Output

Messages

Notifications

Showing rows: 1 to 6

Page No:

	enrollment_id [PK] integer	student_id integer	status enrollment_status
1	2	2	active
2	5	2	active
3	1	1	completed
4	3	1	completed
5	4	1	completed
6	6	3	completed

ER-діаграма

